Created Feb 8Feb 8

==========
Integrate tomcat in Jenkins Pipeline

Install plugin : deploy to container

Restart Jenkins

systemctl restart Jenkins.service

Manage Jenkins --> Credentials --> System --> Global credentials (unrestricted) --> Add credentials --> Username:tomcat, password: root123456, id = tomcatcreds

Open Pipeline Syntax
---
select deploy:Deploy war/ear to a container
WAR/EAR files = **/*.war
Context path = myapp
Add container --> select tomcat 9
credentials = tomcatcreds
Tomcat URL = http://13.126.17.44:8080/

Generate
=========

Tomcat integration Pipeline

```
pipeline {
    agent any
    stages {
        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-new.git&#39;
            }
        }
        stage('compile') {
            steps {
                sh 'mvn compile'
            }
        }
        stage('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('artifact') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('deploy') {
            steps {
                deploy adapters: [tomcat9(credentialsId: 'tomcatcreds', path: '', url: 'http://13.126.17.44:8080/&#39;)], contextPath: 'myapp', war: '**/*.war'
            }
        }
```

```
    }
}
```

Upload artifacts to S3
======================

Instal plugins = S3 publisher

RESTART the Jenkins

Manage Jenkins --> System --> S3 Profile --> Name = s3creds, Access Key and Secret Key and Test

Create a New Build job

Use Pipeline Script S3upload and fill all details

```
pipeline {
    agent {
        label 'slave1'
    }
    stages {
        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-new.git&#39;
            }
        }
        stage('compile') {
            steps {
                sh 'mvn compile'
            }
        }
        stage('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('artifact') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Upload to S3') {
            steps {
                s3Upload consoleLogLevel: 'INFO', dontSetBuildResultOnFailure: false, dontWaitForConcurrentBuildCompletion: false, entries: [[bucket: 'jenkins-artifacts-demo-test', excludedFile: '', flatten: false, gzipFiles: false, keepForever: false, managedArtifacts: false, noUploadOnFailure: false, selectedRegion: 'ap-south-1', showDirectlyInBrowser: false, sourceFile: '**/*.war', storageClass: 'STANDARD', uploadFromSlave: false, useServerSideEncryption: true]], pluginFailureResultConstraint: 'FAILURE', profileName: 's3creds', userMetadata: []
            }
        }
        stage('deploy') {
            steps {
                deploy adapters: [tomcat9(credentialsId: 'tomcatcreds', path:
```

```
'', url: 'http://13.126.17.44:8080/&#39;)], contextPath: 'myapp', war:
'**/*.war'
            }
        }

    }
}
```

```
====================================
SONARQUBE
====================================

====== TAKE SonarQube.sh to install ======


SETUP: Launch another EC2 instance - Amazon linux 2 - t2.medium is must , it
will not run on micro

============ script ========================
#! /bin/bash
#Launch an instance t2.medium, port 9000
cd /opt/
wget
https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-8.9.6.50800.zi
p
unzip sonarqube-8.9.6.50800.zip
amazon-linux-extras install java-openjdk11 -y
useradd sonar
chown sonar:sonar sonarqube-8.9.6.50800 -R
chmod 777 sonarqube-8.9.6.50800 -R
su - sonar
# use the below command manually after installation
#sh /opt/sonarqube-8.9.6.50800/bin/linux-x86-64/sonar.sh start
#echo "user=admin & password=admin"

================================================

Once installed manually type below command

sh /opt/sonarqube-8.9.6.50800/bin/linux-x86-64/sonar.sh start

dont run this - echo "user=admin & password=admin"

==================

==============================
If you have stopped the sonar, start the sonar manually
su - sonar
sh /opt/sonarqube-8.9.6.50800/bin/linux-x86-64/sonar.sh start
==============================

Open http://IP:9000

username: admin,
Password : admin



Add project --> Manually --> project key =boomapp --> generate token--> boomapp
--> click on maven

6c5885cc23041128afecfd2bc764dc7603adebc1
```

In Jenkins: Add plugin,
======================

SonarQube Scanner,
Maven Integration plugins
Sonar Scanner Quality Gates
RESTART JENKINS

Go to Credentials

--> Kind= Secret text , secret = c7fc89bc61e79ad060081f980aa3b0a40dfe9115 (sonar project key)
id --> sonar , description = sonar

Go to System

--> SonarQube servers --> Enable Environment Variables --> Add SonarQube Server
     Name = SonarQube
     URL = http://3.110.190.217:9000
     Server authentication token = select token - sonar

Go to Tools

--> SonarQube Scanner installations , Name = sonarscanner --> dont click add installer

==============================

```
pipeline {
    agent any
    stages {
        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-new.git&#39;
            }
        }
        stage('compile') {
            steps {
                sh 'mvn compile'
            }
        }
        stage('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn org.sonarsource.scanner.maven:sonar-maven-plugin:3.7.0.1746:sonar'
                }
            }
        }
        stage('artifact') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
```

```
}
```

OWASP Dependency-Check is a tool that identifies project dependencies and checks if there are any known, publicly disclosed, vulnerabilities. It can be used in various software development contexts to enhance the security of applications by identifying and alerting developers about vulnerable components that may be included in their projects.

Go to Plugins and install OWASP Dependency-Check and restart

Go to Tools --> Dependency-Check installations --> Name = DC --> install automatically --> Save

OWASP Integration to Jenkins

Go to Manage Jenkins --> Tools -->Dependency-Check installations --> Add
Name = DC
Install automatically
Add installer --> install from GitHub

```
        stage('OWASP Dependency Check'){
            steps {
                dependencyCheck additionalArguments: '--scan ./', 
odcInstallation: 'DC'
                dependencyCheckPublisher pattern: '**/dependency-check-
report.xml'
            }
        }
        stage('Sonar Quality Gate Scan'){
            steps {
                timeout(time: 15, unit:"MINUTES"){
                    waitForQualityGate abortPipeline: false
                }

            }
        }
```